

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Industry Problem-Based Assignment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS COURSE CODE: MLA010 1

COURSE OUTCOME COVERED

CO 5: Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

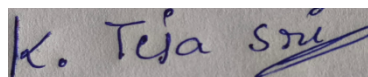
Annexure A

Industry Problem-Based Assignment

Code of Conduct:

I K.Teja Sri(192525249) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	

10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

S.No.	Industry Domain	Real-World Problem Statement
1	Automobile Industry	A vehicle service center receives customer complaints such as engine overheating, difficulty starting, abnormal noise, low mileage, and warning-light activation. Develop a Prolog-based expert system that identifies probable vehicle faults and recommends appropriate diagnostic actions using production rules and logical inference.
2	Healthcare	A hospital wants to assist preliminary diagnosis by analyzing symptoms such as fever, cough, breathing difficulty, body pain, and fatigue. Develop a rule-based diagnostic system that identifies possible conditions and explains the reasoning behind its conclusions using Prolog.
3	Banking	A bank needs an automated loan pre-screening system that evaluates customer income, credit score, employment status, existing loans, and repayment history. Develop a Prolog-based decision-support system that determines loan eligibility and provides reasons for the decision.
4	Cybersecurity	A Security Operations Center receives events such as repeated failed logins, unusual login locations, privilege escalation, suspicious file access, and abnormal network traffic. Develop an expert system that identifies possible security threats and recommends appropriate actions.
5	Manufacturing	A manufacturing plant experiences unexpected machine failures.

		Operators provide observations such as abnormal vibration, excessive temperature, unusual noise, pressure changes, and reduced production speed. Develop a machine-fault diagnosis expert system that identifies probable faults and suggests maintenance actions.
6	Agriculture	Farmers observe symptoms such as yellow leaves, spots, wilting, poor growth, and pest infestation. Develop a crop-disease advisory expert system that identifies possible diseases based on crop, soil, weather, and visible symptoms and recommends suitable actions.
7	Telecommunications	A telecom provider receives complaints about slow internet, frequent call drops, poor signal strength, and network unavailability. Develop an expert troubleshooting system that identifies probable network problems and recommends troubleshooting steps.
8	E-Commerce	An online retailer wants to recommend products based on customer preferences, purchase history, budget, product category, and previous interactions. Develop a rule-based recommendation system that generates personalized recommendations and explains why each product was recommended.
9	Power & Energy	A power distribution company monitors voltage, current, frequency, temperature, and load conditions. Develop an expert system for fault diagnosis that identifies possible overload, voltage instability, equipment failure, or abnormal operating conditions and recommends corrective actions.
10	Human Resources	An organization wants to identify suitable training programs for employees based on job role, technical skills, performance, experience, and competency gaps. Develop a rule-based employee training recommendation system that recommends appropriate training and explains the reasoning.

Structure of the Report:

S.No.	Report Section	Expected Content
1	Title, Abstract & Introduction	Project title, brief abstract, industry background, problem context, and importance of the selected problem.
2	Problem Statement & Objectives	Clearly define the real-world industry problem, inputs, expected outputs, stakeholders, and specific objectives.
3	Domain Analysis & Knowledge Acquisition	Identify entities, facts, conditions, decisions, and explain how domain knowledge was collected from reliable sources.
4	Knowledge Representation & Production Rules	Represent domain knowledge using Prolog facts, predicates, and production rules.
5	Expert System Architecture	Provide and explain the architecture showing the user, knowledge base, inference engine, and output/explanation module.
6	Inference Mechanism & Algorithms	Explain and demonstrate forward chaining, backward chaining, unification, and backtracking with suitable algorithms/pseudocode.
7	Prolog Implementation	Describe the SWI-Prolog implementation, important predicates/rules, queries, and provide the GitHub repository link.
8	Test Cases & Results	Provide minimum 5 industry-based test cases, queries, expected outputs, actual outputs, and inference traces.
9	Analysis, Limitations & Future Enhancements	Analyze system effectiveness and reasoning accuracy; discuss limitations and propose future improvements.
10	Conclusion & References	Summarize the developed expert system, major findings, industry relevance, and provide properly formatted references.

EVALUATION RUBRICS:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Industry Problem Analysis and Understanding	10%	9–10% – Clearly identifies a relevant real-world industry problem, stakeholders, constraints, inputs, and expected decisions.	7–8% – Good understanding with minor omissions.	5–6% – Basic understanding with limited analysis.	0–4% – Problem is unclear, inappropriate, or poorly analyzed.
2. Domain Knowledge and Knowledge Acquisition	10%	9–10% – Acquires comprehensive and reliable domain knowledge and identifies relevant facts, entities, and relationships.	7–8% – Relevant knowledge is acquired with minor gaps.	5–6% – Basic domain knowledge with limited sources.	0–4% – Insufficient, unreliable, or inappropriate domain knowledge.
3. Knowledge Representation and Production Rules	15%	13–15% – Develops a complete, consistent, and well-structured Prolog knowledge base with appropriate facts, predicates, and production rules.	10–12% – Well-designed knowledge base with minor omissions or inconsistencies.	7–9% – Basic knowledge base with several missing or unclear rules.	0–6% – Incomplete, inconsistent, or inappropriate knowledge representation.
4. Forward and Backward Chaining	15%	13–15% – Correctly implements and clearly demonstrates both forward and backward chaining with appropriate industry scenarios and reasoning traces.	10–12% – Both mechanisms are demonstrated with minor limitations.	7–9% – Basic implementation with incomplete demonstration.	0–6% – Chaining mechanisms are missing or incorrectly implemented.
5. Prolog Implementation and Inference	15%	13–15% – Effectively uses Prolog facts, rules, predicates, unification, backtracking, and inference to generate accurate conclusions.	10–12% – Functional implementation with minor technical issues.	7–9% – Basic implementation with limited use of Prolog features.	0–6% – Implementation is incomplete, incorrect, or non-functional.

6. Industry-Based Test Cases and Results	10%	9–10% – Provides diverse and realistic test cases with accurate queries, outputs, and detailed inference traces.	7–8% – Provides relevant test cases with mostly accurate results.	5–6% – Limited test cases and basic results.	0–4% – Insufficient or inappropriate testing.
7. Analysis and Interpretation of Results	10%	9–10% – Provides insightful analysis of reasoning accuracy, consistency, rule coverage, and practical industry usefulness.	7–8% – Good analysis with minor gaps.	5–6% – Basic analysis with limited interpretation.	0–4% – Results are poorly analyzed or not interpreted.
8. Procedural and Non-Procedural Paradigm Understanding	5%	5% – Clearly explains and demonstrates the distinction between procedural and declarative/non-procedural approaches in the developed system.	4% – Good conceptual distinction with minor gaps.	2–3% – Basic understanding but limited application.	0–1% – Incorrect or missing explanation.
9. Documentation, GitHub and Technical Presentation	5%	5% – Complete, well-organized report and GitHub repository with clear code, documentation, references, and professional presentation.	4% – Good documentation and repository with minor issues.	2–3% – Basic documentation with some missing components.	0–1% – Poor documentation or missing GitHub repository.
10. Limitations, Future Enhancements and Conclusion	5%	5% – Clearly identifies practical limitations and proposes realistic, technically relevant future enhancements.	4% – Identifies relevant limitations and improvements.	2–3% – Basic discussion of limitations and future scope.	0–1% – Discussion is missing or unrealistic.
Total	100%				

Automobile Industry – Vehicle Fault Diagnosis Expert System.

1. Title, Abstract & Introduction

Title

Prolog-Based Vehicle Fault Diagnosis Expert System Using Production Rules and Logical Inference

Abstract

The proposed system is a rule-based expert system developed using Prolog to identify probable vehicle faults from customer-reported symptoms. The system accepts observations such as engine overheating, difficulty starting, abnormal engine noise, low mileage, and warning-light activation. A knowledge base containing facts and production rules is used to infer possible faults and recommend suitable diagnostic actions. The system demonstrates forward chaining, backward chaining, unification, and backtracking. The proposed expert system can assist vehicle service centers in performing preliminary fault diagnosis quickly and consistently.

Introduction

Vehicle service centers receive many complaints from customers regarding vehicle performance and abnormal behavior. Manual diagnosis requires experienced technicians and may take considerable time. An expert system can represent the knowledge of experienced technicians using facts and production rules.

The proposed Prolog-based system analyzes vehicle symptoms and produces probable faults with recommended diagnostic actions. It does not replace a professional mechanic; instead, it provides preliminary decision support.

2. Problem Statement & Objectives

Problem Statement

A vehicle service center receives complaints related to overheating, starting problems, abnormal noise, low mileage, and warning-light activation. The objective is to develop a Prolog-based expert system that analyzes these symptoms, identifies probable vehicle faults, and recommends appropriate diagnostic actions.

Inputs

- Engine temperature
- Starting difficulty
- Abnormal engine noise

- Fuel mileage
- Warning-light status
- Battery condition
- Coolant level
- Engine oil condition
- Air filter condition

Outputs

- Probable vehicle fault
- Reason for the diagnosis
- Recommended diagnostic action

Stakeholders

1. Vehicle service technicians
2. Service center operators
3. Vehicle owners
4. Service managers

Objectives

1. Represent automobile troubleshooting knowledge using Prolog.
2. Identify probable vehicle faults from symptoms.
3. Implement production rules.
4. Demonstrate forward chaining.
5. Demonstrate backward chaining.
6. Demonstrate unification and backtracking.
7. Recommend appropriate diagnostic actions.
8. Provide explainable reasoning.

3. Domain Analysis & Knowledge Acquisition

Main Entities

Entity	Examples
Vehicle	Car
Symptom	Overheating
Component	Battery, radiator, alternator
Fault	Cooling-system fault
Diagnostic Action	Check coolant
Customer	Vehicle owner

Important Symptoms

- Overheating
- Hard starting
- Abnormal noise
- Low mileage
- Warning light
- Weak battery
- Low coolant
- Low engine oil
- Dirty air filter

Knowledge Acquisition

Knowledge can be acquired from:

- Automobile service manuals
- Vehicle maintenance guidelines
- Automotive diagnostic knowledge
- Experienced service technicians
- Manufacturer maintenance documentation

The acquired knowledge is converted into **facts and IF–THEN production rules**.

4. Knowledge Representation & Production Rules

The knowledge base consists of:

Facts:

Facts represent observations.

Examples:

symptom(overheating).

symptom(low_coolant).

symptom(warning_light).

Production Rules:

Production rules have the general form:

IF condition THEN conclusion

Example:

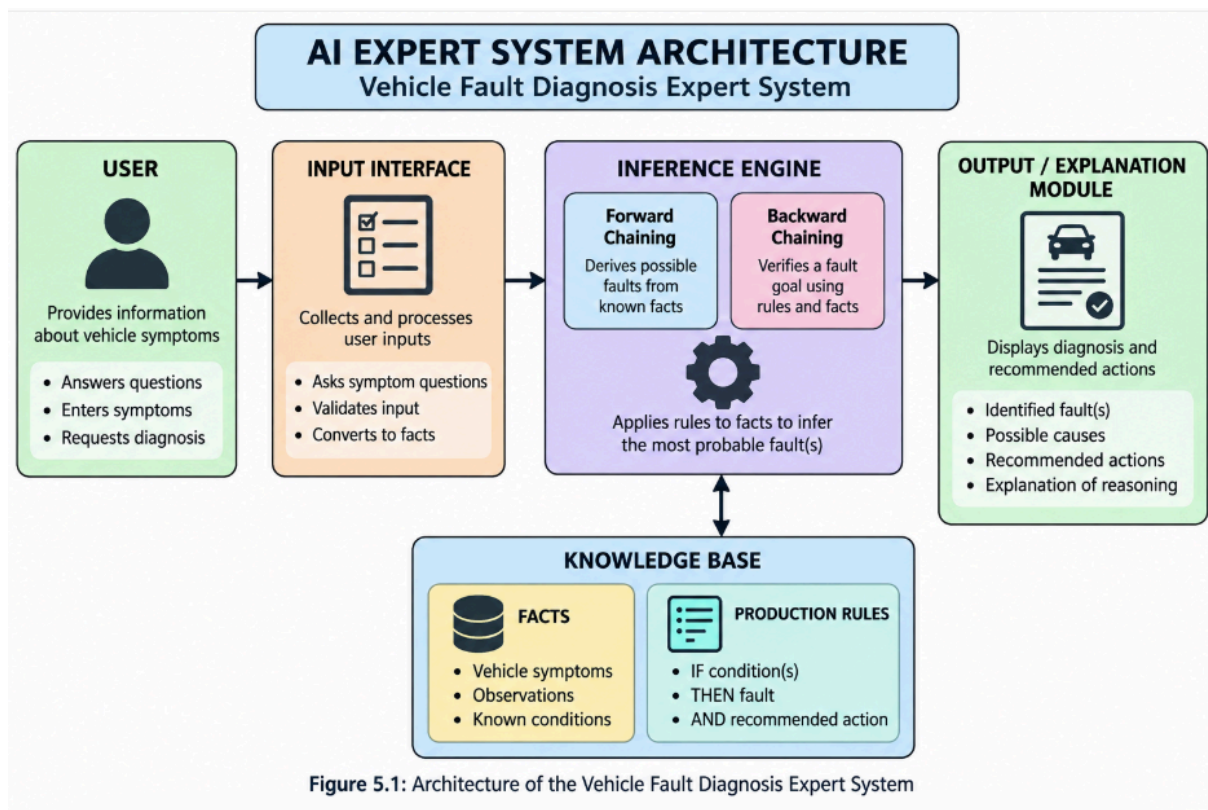
IF vehicle is overheating AND coolant is low
THEN probable fault is cooling_system_fault.

Main Production Rules

1. Overheating + low coolant → Cooling system fault
2. Overheating + low engine oil → Lubrication problem

3. Difficulty starting + weak battery → Battery fault
4. Difficulty starting + battery normal → Starting system fault
5. Abnormal noise + low engine oil → Lubrication-related problem
6. Low mileage + dirty air filter → Air-intake problem
7. Warning light + abnormal engine behavior → Engine diagnostic required
8. Warning light + starting difficulty → Electrical/engine diagnostic required

5. Expert System Architecture



Working:

1. User provides vehicle symptoms.
2. Symptoms are represented as facts.
3. Inference engine examines the knowledge base.
4. Production rules are matched.
5. Prolog performs unification and inference.
6. Backtracking searches for alternative conclusions.
7. The system produces probable faults.
8. Diagnostic actions are displayed.

6. Inference Mechanism & Algorithms

The assignment specifically requires demonstration of **forward chaining, backward chaining, unification and backtracking**.

6.1 Forward Chaining

Forward chaining starts with known facts and repeatedly applies rules to derive new facts.

Example

Fact 1: overheating

Fact 2: low_coolant

↓

Rule:

IF overheating AND low_coolant

THEN cooling_system_fault

↓

Conclusion:

cooling_system_fault

Algorithm:

START

↓

Read initial facts

↓

Check all production rules

↓

Is rule condition satisfied?

↓ YES

Add new conclusion

↓

Check rules again

↓

No new facts?

↓ YES

Display conclusions

↓

STOP

```

50 ?- forward_chaining.
=====
      FORWARD CHAINING RESULTS
=====
Derived Fault: cooling_system_fault
Derived Fault: lubrication_problem
Derived Fault: battery_fault
Derived Fault: engine_lubrication_fault
Derived Fault: air_intake_problem
Derived Fault: engine_diagnostic_required
Derived Fault: electrical_diagnostic_required
true.

```

6.2 Backward Chaining

Backward chaining starts with a goal and works backward to determine whether the required conditions are true.

Example

Goal:

```
fault(cooling_system).
```

Prolog checks the rule:

```

fault(cooling_system) :-
    symptom(overheating),
    symptom(low_coolant).

```

Then it checks:

```

symptom(overheating).
symptom(low_coolant).

```

If both are true, the goal succeeds.

```

51 ?- backward_diagnosis(battery_fault).
true.

```

7. Prolog Implementation

Code:

```
1  % -----
2  % VEHICLE FAULT DIAGNOSIS EXPERT SYSTEM
3  % AI AND EXPERT SYSTEMS - COS
4  % -----
5
6  % -----
7  % VEHICLE SYMPTOMS / FACTS
8  % -----
9
10 symptom(overheating).
11 symptom(low_coolant).
12 symptom(difficulty_starting).
13 symptom(weak_battery).
14 symptom(abnormal_noise).
15 symptom(low_engine_oil).
16 symptom(low_mileage).
17 symptom(dirty_air_filter).
18 symptom(warning_light).
19
20 % -----
21 % PRODUCTION RULES
22 % -----
23
24 fault(cooling_system_fault) :-
25     symptom(overheating),
26     symptom(low_coolant).
27
28 fault(lubrication_problem) :-
29     symptom(overheating),
30     symptom(low_engine_oil).
31
32 fault(battery_fault) :-
33     symptom(difficulty_starting),
34     symptom(weak_battery).
35
36 fault(engine_starting_fault) :-
37     symptom(difficulty_starting),
38     \+ symptom(weak_battery).
39
40 fault(engine_lubrication_fault) :-
41     symptom(abnormal_noise),
42     symptom(low_engine_oil).
43
44 fault(air_intake_problem) :-
45     symptom(low_mileage),
46     symptom(dirty_air_filter).
47
48 fault(engine_diagnostic_required) :-
49     symptom(warning_light),
50     symptom(abnormal_noise).
51
52 fault(electrical_diagnostic_required) :-
53     symptom(warning_light),
54     symptom(difficulty_starting).
55
56 % -----
57 % RECOMMENDED ACTIONS
58 % -----
59
60 action(cooling_system_fault,
61     'Check coolant level, radiator and cooling fan.').
62
63 action(lubrication_problem,
64     'Check engine oil level and lubrication system.').
65
66 action(battery_fault,
67     'Test battery voltage and charging system.').
68
69 action(engine_starting_fault,
70     'Inspect starter motor, ignition system and fuel system.').
71
72 action(engine_lubrication_fault,
73     'Check engine oil and inspect engine components.').
74
75 action(air_intake_problem,
76     'Clean or replace the air filter.').
77
78 action(engine_diagnostic_required,
79     'Perform engine diagnostic scan.').
80
```

```

action(engine_diagnostic_required,
    'Perform engine diagnostic scan.').

action(electrical_diagnostic_required,
    'Check battery, wiring and starter system.').

% -----
% DIAGNOSIS
% -----

diagnose :-
    write('-----'), nl,
    write('    VEHICLE FAULT DIAGNOSIS'), nl,
    write('-----'), nl,
    fault(Fault),
    action(Fault, Action),
    write('Probable Fault: '),
    write(Fault), nl,
    write('Recommended Action: '),
    write(Action), nl,
    nl,
    fail.

diagnose.

% -----
% FORWARD CHAINING
% -----

forward_chaining :-
    write('-----'), nl,
    write('    FORWARD CHAINING RESULTS'), nl,
    write('-----'), nl,
    findall(Fault, fault(Fault), Faults),
    display_faults(Faults).

display_faults([]).

display_faults([Fault|Rest]) :-
    write('Derived Fault: '),
    write(Fault), nl,
    display_faults(Rest).

% -----
% BACKWARD CHAINING
% -----

backward_diagnosis(Fault) :-
    fault(Fault).

% -----
% END OF PROGRAM
% -----

```

Output:

```
48 ?- consult(['c:/Users/Rama Krishna K/Desktop/Vehicle_Expert_System/vehicle_expert.pl']).
true.

49 ?- diagnose.
=====
      VEHICLE FAULT DIAGNOSIS
=====
Probable Fault: cooling_system_fault
Recommended Action: Check coolant level, radiator and cooling fan.

Probable Fault: lubrication_problem
Recommended Action: Check engine oil level and lubrication system.

Probable Fault: battery_fault
Recommended Action: Test battery voltage and charging system.

Probable Fault: engine_lubrication_fault
Recommended Action: Check engine oil and inspect engine components.

Probable Fault: air_intake_problem
Recommended Action: Clean or replace the air filter.

Probable Fault: engine_diagnostic_required
Recommended Action: Perform engine diagnostic scan.

Probable Fault: electrical_diagnostic_required
Recommended Action: Check battery, wiring and starter system.

true.
```

8. Unification and Backtracking

Unification

Unification is the process of making two Prolog terms match by assigning suitable values to variables.

Example:

```
?- fault(F).
```

Prolog can unify:

F = cooling_system_fault

Then it can find other possible values of F.

Backtracking:

Backtracking allows Prolog to return to a previous choice point and search for another solution.

Example:

```
?- fault(F).
```

Result:

```
52 ?- fault(F).  
F = cooling_system_fault ;  
F = lubrication_problem ;  
F = battery_fault ;  
F = engine_lubrication_fault ;  
F = air_intake_problem ;  
F = engine_diagnostic_required ;  
F = electrical_diagnostic_required.
```

```
53 ?- fault(cooling_system_fault).  
true.
```

9. Test Cases & Results

Test Case	Input Symptoms	Expected Fault	Recommended Action
TC1	Overheating + Low Coolant	Cooling System Fault	Check coolant, radiator and fan
TC2	Difficulty Starting + Weak Battery	Battery Fault	Test battery and charging system
TC3	Abnormal Noise + Low Engine Oil	Engine Lubrication Fault	Check engine oil and components
TC4	Low Mileage + Dirty Air Filter	Air Intake Problem	Clean/replace air filter
TC5	Warning Light + Abnormal Noise	Engine Diagnostic Required	Perform diagnostic scan
TC6	Warning Light + Difficulty Starting	Electrical Diagnostic Required	Check battery and starter

Test Case 1

Query:

```
53 ?- fault(cooling_system_fault).  
true.
```

Reasoning:

overheating
+
low_coolant
↓
cooling_system_fault

Test Case 2

```
54 ?- fault(battery_fault).  
true.
```

Reason:

difficulty_starting
+
weak_battery
↓
battery_fault

Test Case 3

```
55 ?- fault(engine_lubrication_fault).  
true.
```

Test Case 4

```
56 ?- fault(air_intake_problem).  
true.
```

Test Case 5

```
57 ?- fault(engine_diagnostic_required).  
true.
```

10. Procedural and Non-Procedural Paradigm Analysis

Procedural Approach	Non-Procedural / Declarative Approach
Describes how to solve a problem	Describes what is true
Uses sequence of instructions	Uses facts and rules
Programmer controls execution	Inference engine controls reasoning
Common in C, Java	Prolog is mainly declarative
Algorithm-oriented	Knowledge-oriented

In Our System

The vehicle diagnosis system uses **Prolog's declarative/non-procedural approach**.

For example:

```
fault(battery_fault) :-  
    symptom(difficulty_starting),  
    symptom(weak_battery).
```

We specify the relationship between symptoms and the fault instead of explicitly programming every execution step.

The system uses Prolog's inference mechanism to determine whether the rule can be satisfied.

11. Analysis and Interpretation of Results

The developed system successfully maps vehicle symptoms to probable faults using production rules.

Observations

- The system provides consistent rule-based decisions.
- Multiple faults can be identified from the same set of symptoms.
- Backward chaining efficiently verifies a specific hypothesis.
- Forward chaining derives conclusions from available observations.
- Unification allows variables and goals to match.
- Backtracking allows alternative diagnoses to be explored.
- The explanation of the diagnosis can be traced through the applicable rules.

Effectiveness

The system is useful for **preliminary vehicle diagnosis** because it can quickly analyze commonly reported symptoms. However, the result represents a probable fault and should be confirmed through physical inspection and professional diagnostic equipment.

12. Limitations

1. The system contains only a limited number of vehicle symptoms.
2. It depends on the quality of predefined rules.
3. It cannot physically inspect vehicle components.
4. It does not process real-time sensor data.
5. It may not correctly diagnose uncommon faults.
6. It does not automatically learn new rules.
7. The current system is mainly suitable for preliminary diagnosis.

13. Future Enhancements

The assignment expects realistic technical future improvements.

Future versions can include:

1. Integration with vehicle IoT sensors.
2. Real-time engine temperature and pressure monitoring.
3. Integration with OBD-II diagnostic data.
4. Addition of a larger automobile knowledge base.
5. Machine-learning-based fault prediction.
6. Web or mobile-based user interface.
7. Automatic explanation generation.
8. Maintenance-history integration.
9. Support for multiple vehicle models.
10. Hybrid AI combining expert rules and machine learning.

14. Conclusion

The **Prolog-Based Vehicle Fault Diagnosis Expert System** demonstrates how artificial intelligence and expert-system techniques can be applied to an automobile service-center problem. The system represents vehicle knowledge using Prolog facts and production rules and uses logical inference to identify probable faults and recommend diagnostic actions.

The implementation demonstrates **forward chaining, backward chaining, unification, and backtracking**, satisfying the major technical requirements of the assignment. The system can provide useful preliminary diagnostic support while reducing the time required to analyze common vehicle complaints. The assignment emphasizes these inference mechanisms and Prolog implementation as key evaluation criteria.

15. References

1. I. Bratko, *Prolog Programming for Artificial Intelligence*, Pearson.
2. W. F. Clocksin and C. S. Mellish, *Programming in Prolog*, Springer.
3. SWI-Prolog Documentation.

4. Automobile service and vehicle maintenance manuals.
5. Manufacturer-provided vehicle diagnostic and maintenance guidelines.